

JVM的体系结构及底层原理

1 | 在需要生成目录的地方输入：[TOC] 然后回车即可。

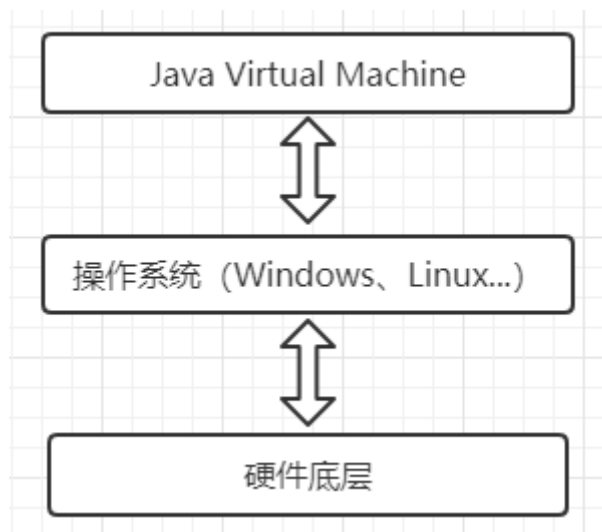
JVM的体系结构及底层原理

- 1.JVM体系结构
- 2.类装载器 ClassLoader
 - 2.1 有哪些类装载器?
 - 2.2 双亲委派和沙箱安全
- 3.本地方法栈 Native Method Stack
 - 本地方法栈 (Native Method Stack) 详解
 - 存储内容
 - 与 Java 虚拟机栈的区别
 - 重要特性
 - 本地方法栈的工作流程
- 4.程序计数器 Program Counter Register
- 5.方法区 Method Area
- 6.栈 Stack
 - 6.1 栈存储什么数据?
 - Java 虚拟机栈 (JVM Stack) 详解
 - 基本概念
 - 栈的核心特性
 - 栈帧 (Stack Frame) 结构
 1. 局部变量表 (Local Variable Table)
 2. 操作数栈 (Operand Stack)
 3. 动态链接 (Dynamic Linking)
 4. 方法返回地址
 - 栈的异常
 - 栈的性能优化
 - 实际应用示例
 - 栈的监控与调优
 - 总结
 - 6.3 栈溢出 StackOverflowError
- 7.栈、堆、方法区的交互关系
- 8.堆 Heap
 - 8.1 堆体系结构
 - 8.2 对象在堆中的生命周期
 - 8.3 Minor GC的过程
 - 8.4 HotSpot虚拟机的内存管理
 - 8.5 永久区
 - 方法区、永久代和元空间的区别
 1. 方法区 (Method Area)
 2. 永久代 (PermGen, Permanent Generation)
 3. 元空间 (Metaspace)
 - 三者的关系
 - 重要变化
 - 相关 JVM 参数
 - 总结
 - 8.6 堆参数调优
 - 8.7 堆溢出 OutOfMemoryError
- 9.GC (Java Garbage Collection)
 - 9.1 GC垃圾收集机制
 - 9.2 GC日志信息详解

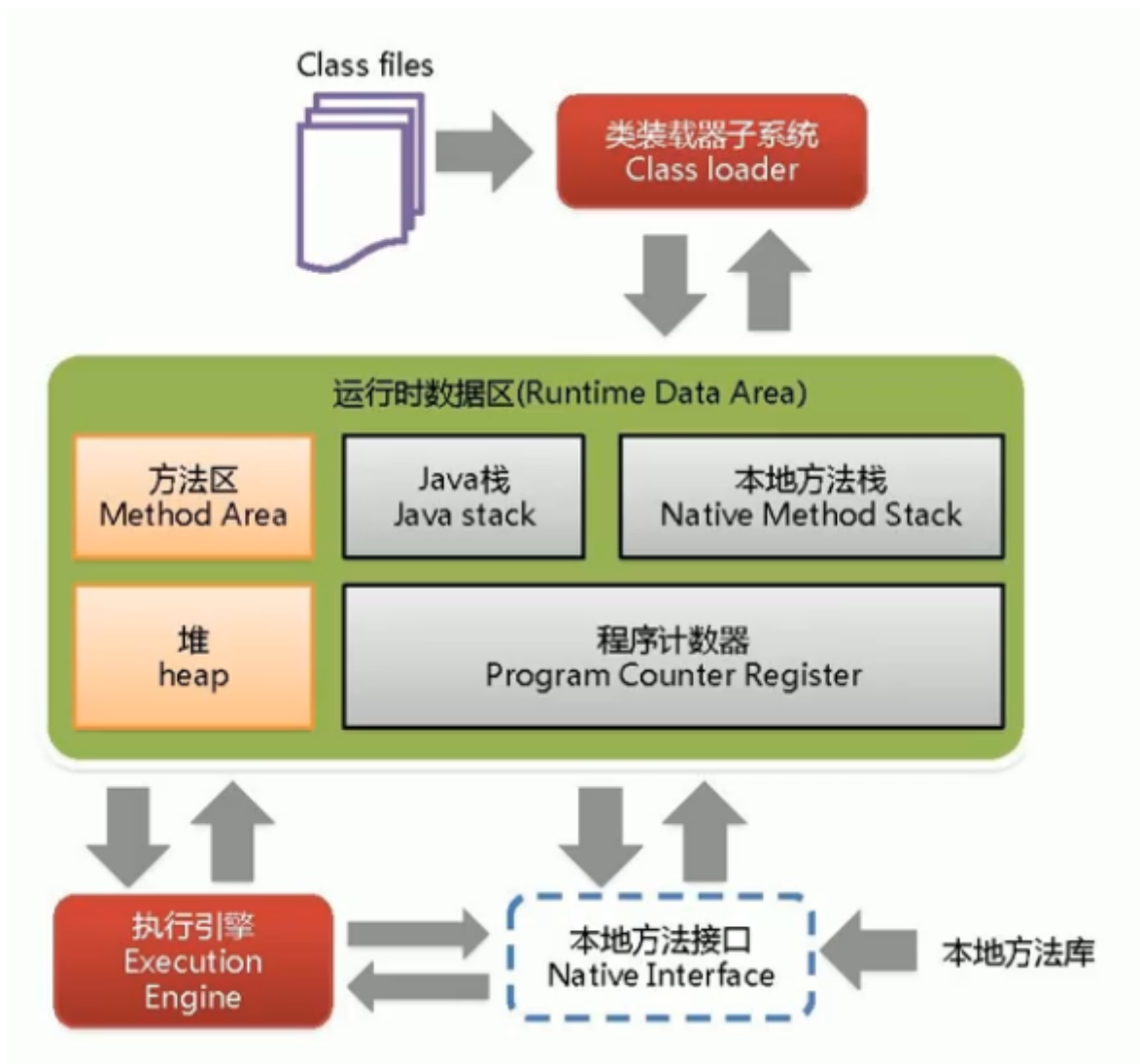
- 9.3 GC四大算法
- 10.如何判断java中对象是否存活?
 - 10.1 引用计数算法
 - 10.2 根搜索方法
- 11.复制算法 (Copying) : 适用于新生代
 - 11.1 原理分析
 - 11.2 优缺点
- 12.标记清除 (Mark-Sweep) : 适用于老年代
 - 12.1 原理分析
 - 12.2 优缺点
- 13.标记压缩 (Mark-Compact) : 适用于老年代
 - 13.1 原理分析
 - 13.2 优缺点
 - 13.3 标记清除压缩 (Mark-Sweep-Compact)
- 14.分代收集算法
- 15.总结
 - 15.1 年轻代 (Young Gen)
 - 15.2 老年代 (Tenure Gen)
- 16.附录.常见面试问题
 - 16.1 GC四种算法哪个好?
 - 16.2 请说出各个垃圾回收算法的优缺点

1.JVM体系结构

首先, 我们来了解 JVM 的位置, 如下图, JVM 是运行在操作系统之上的, 它与硬件没有直接交互。



接下来是 JVM 的整个体系结构图



:

然后，我们根据 JVM 体系结构图来阐述相关的知识点。

线程独占:栈,本地方法栈,程序计数器

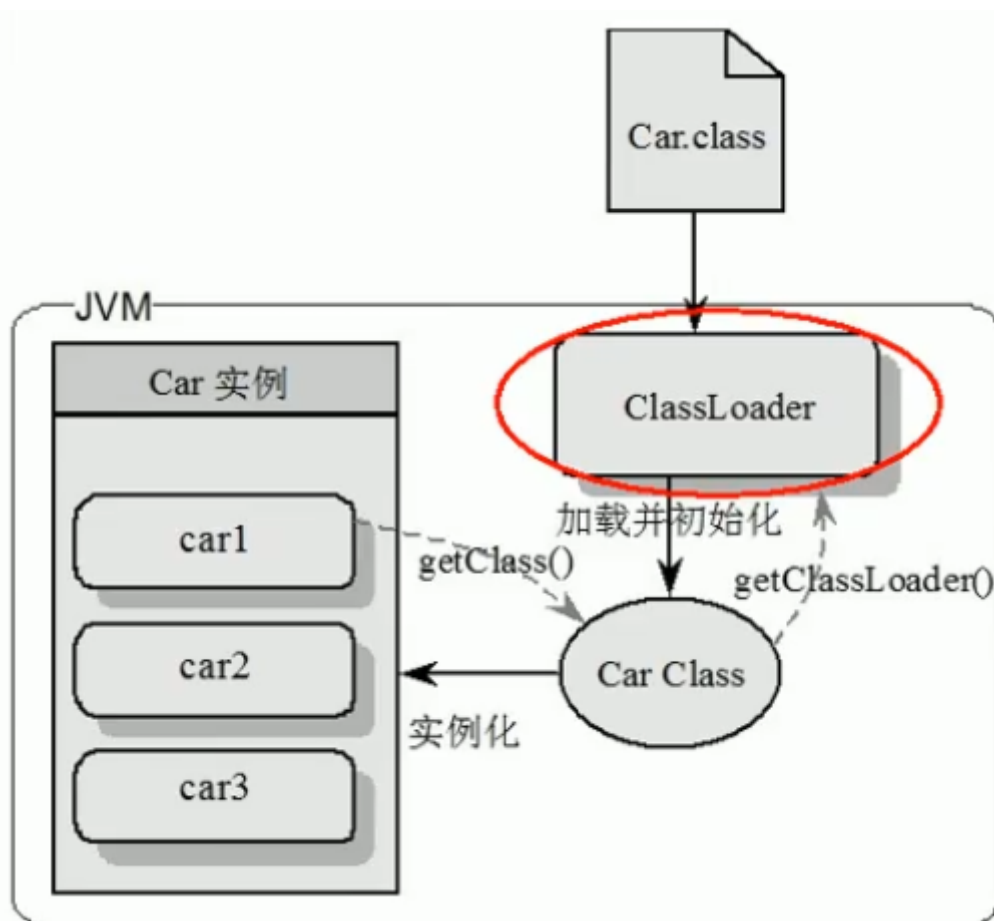
线程共享:堆,方法区

2.类装载器 ClassLoader

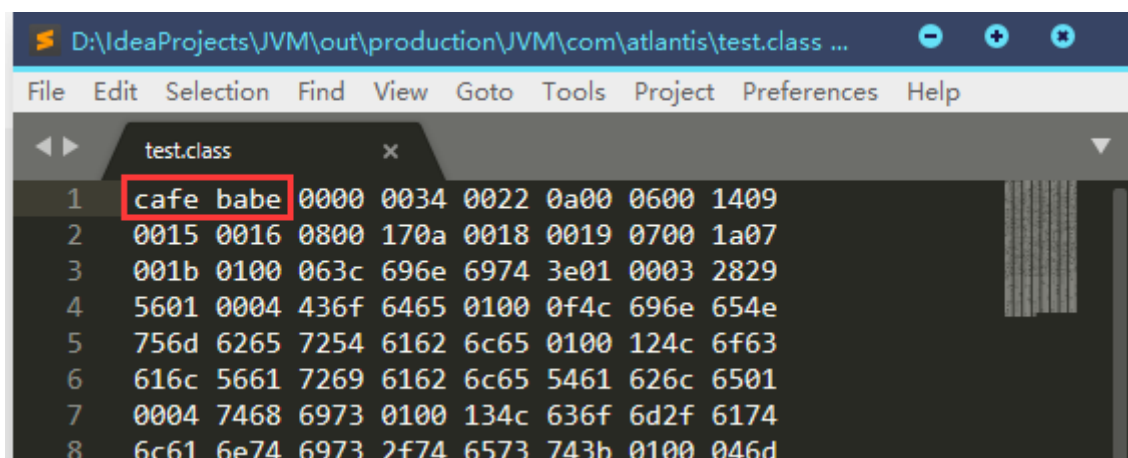
类装载器 `ClassLoader` 是负责加载class文件的，将class文件字节码内容加载到内存中，并将这些内容转换成方法区中的运行时数据结构。`ClassLoader`只负责文件的加载，至于它是否可运行，则由 `Execution Engine`决定。

在这里需要区分一下class与Class。小写的class，是指编译 Java 代码后所生成的以.class为后缀名的字节码文件。而大写的Class，是JDK提供的java.lang.Class，可以理解为封装类的模板。多用于反射场景，例如JDBC中的加载驱动，`Class.forName("com.mysql.jdbc.Driver")`;

接下来我们来观察下图，`Car.class`字节码文件被`ClassLoader`类装载器加载并初始化，在方法区中生成了一个**Car Class**的类模板，而我们平时所用到的实例化，就是在这个类模板的基础上，形成了一个个实例，即`car1`，`car2`。反过来讲，我们可以对某个具体的实例进行**`getClass()`**操作，就可以得到该实例的类模板，即`Car Class`。再接着，我们对这个类模板进行**`getClassLoader()`**操作，就可以得到这个类模板是由哪个类装载器进行加载的。



Tip: 扩展一下, JVM并不仅仅只是通过检查文件后缀名是否是.class来判断是否加载, 最主要的是通过class文件中特定的文件标示, 即下图test.class文件中的cafe babe。

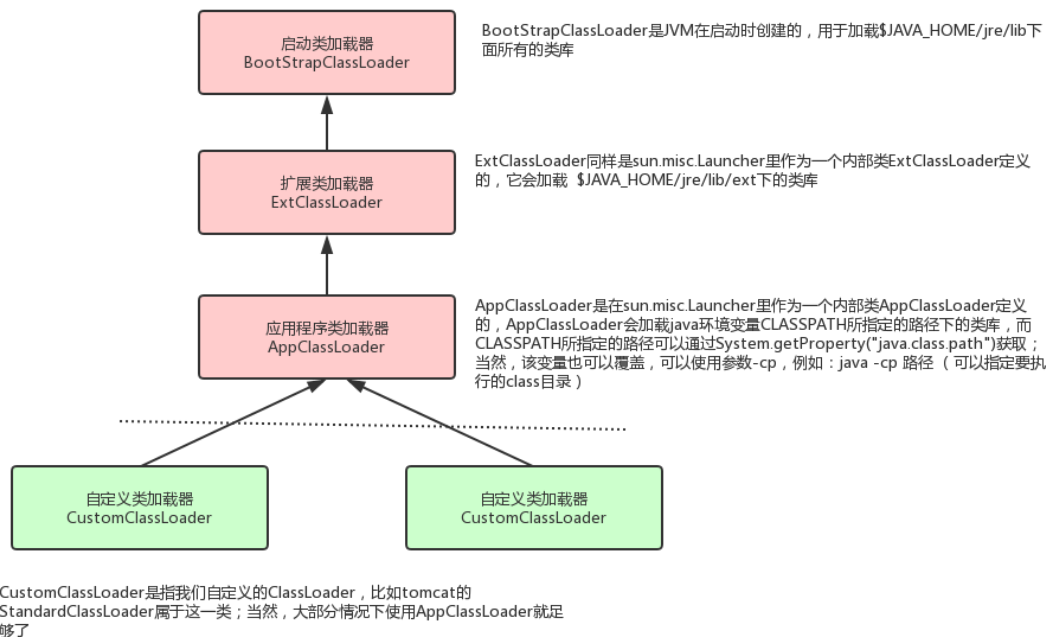


2.1 有哪些类装载器?

(1) 虚拟机自带的加载器

- 启动类加载器 (Bootstrap), 也叫根加载器, 加载`%JAVAHOME%/jre/lib/rt.jar`等*.jar。
- 扩展类加载器 (Extension), 加载`%JAVAHOME%/jre/lib/ext/*.jar`, 例如javax.swing包。
- 应用程序类加载器 (AppClassLoader), 也叫系统类加载器, 加载`%CLASSPATH%`的所有类。

(2) 用户自定义的加载器: 用户可以自定义类的加载方式, 但必须是`java.lang.ClassLoader`的子类。



2.2 双亲委派和沙箱安全

接下来，我们通过下面代码来观察这几个类加载器。首先，我们先看自定义的MyObject，首先通过getClassLoader()获取到的是AppClassLoader，然后getParent()得到ExtClassLoader，再getParent()竟然是null？可能大家会有疑惑，不应该是Bootstrap加载器么？这是因为，BootstrapClassLoader是使用C++语言编写的，Java在加载的时候就成了null。

我们再来看Java自带的Object，通过getClassLoader()获取到的加载器直接就是BootstrapClassLoader，如果要想getParent()的话，因为null值，所以就会报java.lang.NullPointerException空指针异常。

输出中，sun.misc.Launcher是JVM相关调用的入口程序。

那为什么会出现这个情况呢？这就需要我们了解类加载器的加载顺序和机制了，即双亲委派和沙箱安全。

(1) 双亲委派，当一个类收到了类加载请求，它首先不会尝试自己去加载这个类，而是把这个请求委派给父类去完成，因此所有的加载请求都应该传送到启动类加载器中，只有当父类加载器反馈自己无法完成这个请求的时候（在它的加载路径下没有找到所需加载的Class），子类加载器才会尝试自己去加载。

采用双亲委派的一个好处是，比如加载位于rt.jar包中的类java.lang.Object，不管是哪个加载器加载这个类，最终都是委派给顶层的启动类加载器进行加载，确保哪怕使用了不同的类加载器，最终得到的都是同样一个Object对象。

(2) 沙箱安全机制，是基于双亲委派机制上采取的一种JVM的自我保护机制，假设你要写一个java.lang.String的类，由于双亲委派机制的原理，此请求会先交给BootstrapClassLoader试图进行加载，但是BootstrapClassLoader在加载类时首先通过包和类名查找rt.jar中有没有该类，有则优先加载rt.jar包中的类，因此就保证了Java的运行机制不会被破坏，确保你的代码不会污染到Java的源码。

所以，类加载器的加载顺序如下：

当AppClassLoader加载一个class时，它首先不会自己去尝试加载这个类，而是把类加载请求委派给父类加载器ExtClassLoader去完成。

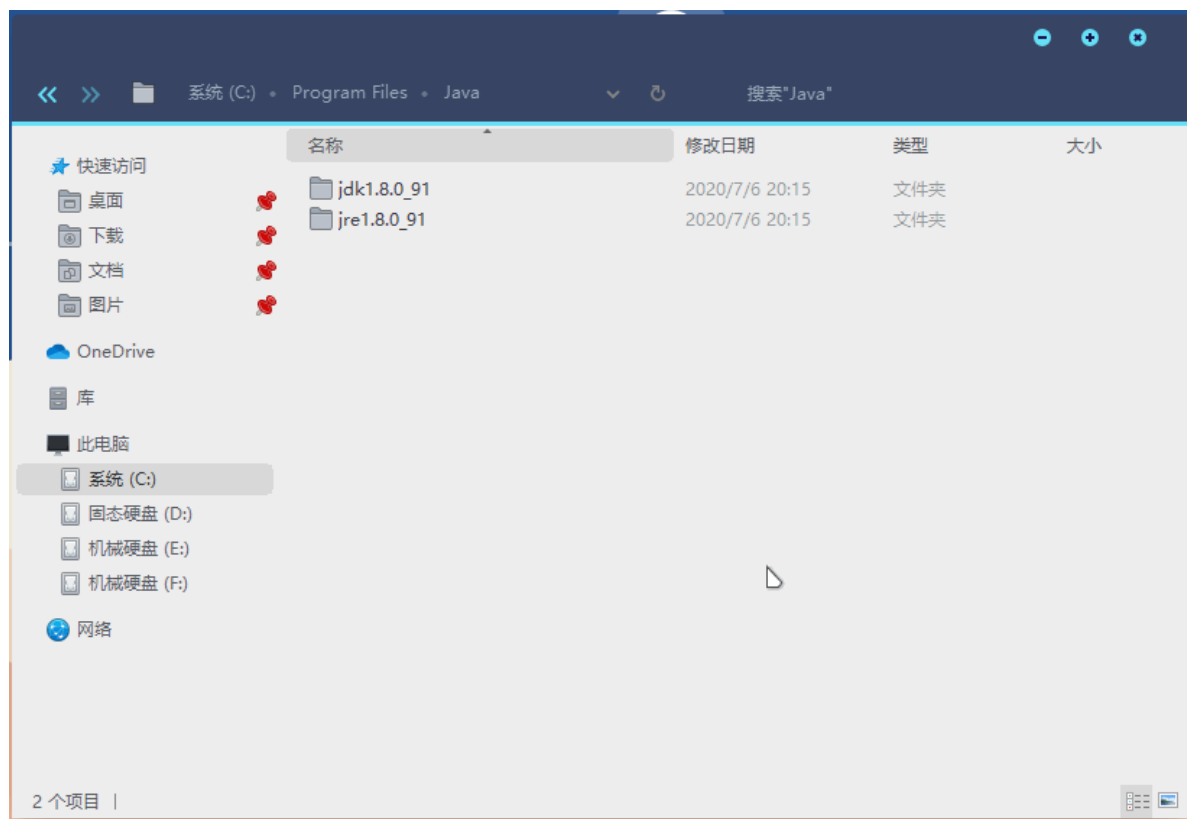
当ExtClassLoader加载一个class时，它首先也不会自己去尝试加载这个类，而是把类加载请求委派给BootstrapClassLoader去完成。

如果BootstrapClassLoader加载失败（例如在\$JAVA_HOME/jre/lib里未查找到该class），会使用

ExtClassLoader来尝试加载。

若**ExtClassLoader**也加载失败，则会使用**AppClassLoader**来加载，如果**AppClassLoader**也加载失败，则会报出异常**ClassNotFoundException**。

Tip: **rt.jar**是什么？做了哪些事？这些暂且不提，那你有没有想过，为什么可以在idea这些开发工具中可以直接去使用String、ArrayList、甚至一些JDK提供的类和方法？观察下面动图就可以知道，原来这些都在rt.jar中定义好了，且直接被启动类加载器进行加载了。



3.本地方法栈 Native Method Stack

本地方法栈（Native Method Stack）详解

本地方法栈是 JVM 运行时数据区的一部分，主要用于支持本地方法（Native Method）的执行。以下是它的主要特点和存储内容：

存储内容

1. 本地方法调用的栈帧

- 存储本地方法的参数
- 保存本地方法的局部变量
- 存储方法调用的返回地址

2. 本地方法执行状态

- 方法调用的上下文信息
- 本地方法执行时的临时数据

与 Java 虚拟机栈的区别

特性	本地方法栈	Java 虚拟机栈
服务对象	为本地方法服务	为 Java 方法服务
实现语言	通常由 C/C++ 实现	由 Java 实现
存储内容	本地方法调用的栈帧	Java 方法调用的栈帧
异常类型	可能抛出本地方法错误	可能抛出 StackOverflowError 或 OutOfMemoryError
内存管理	由本地方法实现管理	由 JVM 管理

重要特性

1. **线程私有**：每个线程都有自己的本地方法栈
2. **可选实现**：JVM 规范不强制要求实现本地方法栈
3. **大小设置**：可以通过-Xss参数设置栈大小
4. **内存溢出**：可能抛出StackOverflowError 或OutOfMemoryError

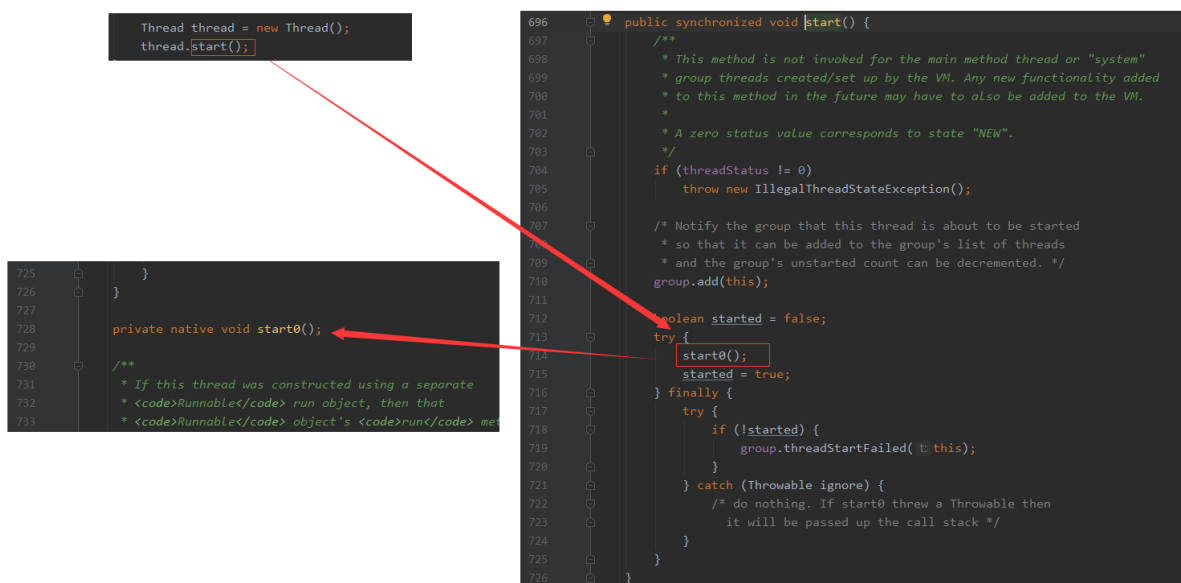
本地方法栈的工作流程

1. Java 代码调用 `native` 方法
2. JVM 在本地方法栈中创建新的栈帧
3. 控制权转移到本地方法实现
4. 本地方法执行期间使用本地方法栈
5. 方法执行完毕，栈帧出栈，控制权返回给 Java 代码

本地方法接口 (Native Interface)，其作用是融合不同的编程语言为 Java 所用，它的初衷是用来融合 C/C++ 程序的，Java 诞生的时候是 C/C++ 流行时期，要想立足，就得调用 C/C++ 程序，于是 Java 就在内存中专门开辟了一块区域处理标记为 `native` 的代码。

而本地方法栈 (Native Method Stack)，就是在 **一个 Stack 中登记这些 native 方法**，然后在 **执行引擎 Execution Engine** 执行时加载本地方法库 **native libraies**。

接下来，我们通过下图的多线程部分源码来理解什么是 **native** 方法。首先我们观察 `start()` 的源码，发现它其实并没有做什么复杂的操作，只是单纯的调用了 `start0()` 这个方法，然后我们去观察 `start0()` 的源码，发现它只是一个使用了 **native关键字** 修饰的一个方法 (`private native void start0();`)，但只有声明却没有具体的实现！。



为什么？我们都知道**Thread**是**Class**关键字修饰的类（**class Thread implements Runnable**），而不是接口。一般来说，类中的方法都要有定义和实现，接口里面才有方法的定义声明。这就是native方法的独特之处，说白了，被native关键字修饰的方法，基本上和我们，甚至和Java 都没啥关系了，因为它要去调用底层操作系统或者第三方语言的库函数，所以我们不需要去考虑它具体是如何实现的。

4.程序计数器 Program Counter Register

程序计数器（Program Counter Register），也叫**PC寄存器**。每个线程启动的时候，都会创建一个PC寄存器。PC寄存器里**保存当前正在执行的JVM指令的地址**。**每一个线程都有它自己的PC寄存器**，也是该线程启动时创建的。

简单来说，PC寄存器就是保存下一条将要执行的指令地址的寄存器，其内容总是指向下一条将被执行指令的地址，这里的地址可以是一个本地指针，也可以是在方法区中相对应于该方法起始指令的偏移量。

每个线程都有一个程序计数器，是线程私有的,就是一个指针，指向方法区中的方法字节码（用来存储指向下一条指令的地址,也即将要执行的指令代码），由执行引擎**Execution Engine**读取下一条指令，是一个非常小的内存空间，几乎可以忽略不计。

这块内存区域很小，它是当前线程所执行的字节码的行号指示器，字节码解释器通过改变这个计数器的值来选取下一条需要执行的字节码指令。

PC寄存器一般用以完成分支、循环、跳转、异常处理、线程恢复等基础功能。不会发生内存溢出（OutOfMemory，OOM）错误。

如果执行的是一个**native**方法，那这个计数器是空的。

5.方法区 Method Area

方法区（Method Area），是供各**线程共享**的运行时内存区域，它**存储了每一个类的结构信息**。例如**运行时常量池**（Runtime Constant Pool）、字段和方法数据、构造函数和普通方法的字节码内容。

上面说的是规范（定义的一种抽象概念），实际在不同虚拟机里实现是不一样的，最典型的就是永久代（PermGen space）和元空间（Meta space）。

1 | 实例变量存在堆内存中，和方法区无关。

6.栈 Stack

栈管运行，堆管存储！ 请熟读并默写全文。（´•ω•）/（´/Д；`）

栈（Stack），也叫栈内存，主管Java程序的运行，在线程创建时创建。其生命期是跟随线程的生命期，是线程私有的，线程结束栈内存也就是释放。

对于栈来说，不存在垃圾回收的问题，只要线程一结束该栈就Over。

6.1 栈存储什么数据？

Java 虚拟机栈（JVM Stack）详解

基本概念

Java 虚拟机栈是线程私有的，它的生命周期与线程相同。每个方法在运行的同时都会创建一个栈帧（Stack Frame）用于存储局部变量表、操作数栈、动态链接、方法出口等信息。每个方法从调用直至执行完成的过程，就对应着一个栈帧在虚拟机栈中入栈到出栈的过程。

栈的核心特性

1. **线程私有**：每个线程都有自己独立的虚拟机栈
2. **后进先出 (LIFO)**：方法的调用和返回遵循后进先出的原则
3. **自动内存管理**：栈帧的创建和销毁由 JVM 自动管理
4. **大小可配**：可以通过-Xss参数设置栈大小

栈帧 (Stack Frame) 结构

1. 局部变量表 (Local Variable Table)

- **存储内容**：**方法参数和局部变量**
- **基本单位**：变量槽 (Variable Slot)
- **存储类型**：
 - 基本数据类型 (boolean, byte, char, short, int, float, long, double)
 - 对象引用 (reference)
 - returnAddress 类型 (指向一条字节码指令的地址)
- **特点**：
 - 局部变量表所需的内存空间在编译期间完成分配
 - 方法运行期间不会改变局部变量表的大小

2. 操作数栈 (Operand Stack)

- **作用**：是栈帧 (Stack Frame) 中的一个后进先出 (LIFO) 的数据结构，用于**存储计算过程中的中间结果和临时变量**
- **特点**：
 - 操作数栈的最大深度在编译期已经确定
 - 32位数据类型占1个栈容量，64位数据类型占2个栈容量
- **操作**：
 - 入栈 (push)
 - 出栈 (pop)
 - 复制 (dup)
 - 交换 (swap)

示例1：简单的加法运算

```
1  javapublic int add() {
2      int a = 10;
3      int b = 20;
4      int c = a + b;
5      return c;
6  }
```

字节码分析：

```

1  java0: bipush      10  // 将10压入操作数栈
2  2: istore_1        // 将栈顶的10存入局部变量表索引1(a)
3  3: bipush          20  // 将20压入操作数栈
4  5: istore_2        // 将栈顶的20存入局部变量表索引2(b)
5  6: iload_1         // 将局部变量表索引1的值(10)压入操作数栈
6  7: iload_2         // 将局部变量表索引2的值(20)压入操作数栈
7  8: iadd            // 弹出栈顶两个int值相加, 结果(30)压入栈顶
8  9: istore_3        // 将栈顶结果30存入局部变量表索引3(c)
9  10: iload_3        // 将局部变量表索引3的值(30)压入操作数栈
10 11: ireturn        // 返回栈顶的int值

```

3. 动态链接 (Dynamic Linking)

- **作用：**将符号引用转换为直接引用
- 代码示例

```

1  javapublic class DynamicLinkingDemo {
2      public static void main(String[] args) {
3          Animal animal = new Dog(); // 多态
4          animal.sayHello();         // 动态绑定
5      }
6  }
7
8  class Animal {
9      public void sayHello() {
10         System.out.println("Animal says hello");
11     }
12 }
13
14 class Dog extends Animal {
15     @Override
16     public void sayHello() {
17         System.out.println("Dog says wang wang");
18     }
19 }
20 动态链接的工作过程
21 编译期: 生成.class文件时, 所有变量和方法引用都作为符号引用保存在.class文件的常量池中
22 类加载: 类加载阶段将符号引用解析为直接引用 (静态解析)
23 运行期: 对于虚方法调用, 在方法第一次执行时解析 (动态解析)

```

- 组成:
 - 运行时常量池的引用
 - 方法返回地址
 - 异常表

4. 方法返回地址

方法返回地址是栈帧 (Stack Frame) 中的一个重要组成部分, 它保存了方法执行完成后应该返回到调用者的位置信息。

方法返回地址是指**方法退出后返回到方法被调用位置的地址**, 它告诉JVM方法执行完成后应该继续执行哪条指令。

- **正常返回：**调用者的PC计数器的值
- **异常返回：**通过异常处理器表确定

- 特点：方法退出时可能执行的操作
 - 恢复上层方法的局部变量表和操作数栈
 - 把返回值压入调用者栈帧的操作数栈
 - 调整PC计数器的值

栈的异常

1. StackOverflowError

- 原因：线程请求的栈深度大于虚拟机所允许的最大深度
- 常见场景：递归调用没有正确的终止条件

```
1  javapublic class StackOverflowExample {  
2      public static void recursiveCall() {  
3          recursiveCall(); // 无限递归  
4      }  
5      public static void main(String[] args) {  
6          recursiveCall();  
7      }  
8  }
```

2. OutOfMemoryError

- 原因：虚拟机栈可以动态扩展，当扩展时无法申请到足够的内存
- 场景：创建过多线程

栈的性能优化

1. 逃逸分析

- 对象在方法内部创建且不会逃逸到方法外部
- JIT 编译器可能会进行栈上分配 (Stack Allocation)

2. 方法内联

- 将方法调用直接替换为方法体
- 减少方法调用的开销

3. 栈帧复用

- 栈帧可以在方法调用之间重用
- 减少栈帧的创建和销毁开销

实际应用示例

```
1  public class StackExample {  
2      public static void main(String[] args) {  
3          int a = 1;  
4          int b = 2;  
5          int result = add(a, b);  
6          System.out.println("Result: " + result);  
7      }  
8  
9      public static int add(int x, int y) {  
10         int sum = x + y;  
11         return sum;  
12     }  
13 }
```

栈的监控与调优

1. 查看栈信息

```
1 bash# 查看线程栈信息
2 jstack <pid>
3
4 # 查看JVM栈大小
5 java -XX:+PrintFlagsFinal -version | grep ThreadStackSize
```

2. 设置栈大小

```
1 bash# 设置栈大小为1MB
2 java -Xss1m YourMainClass
```

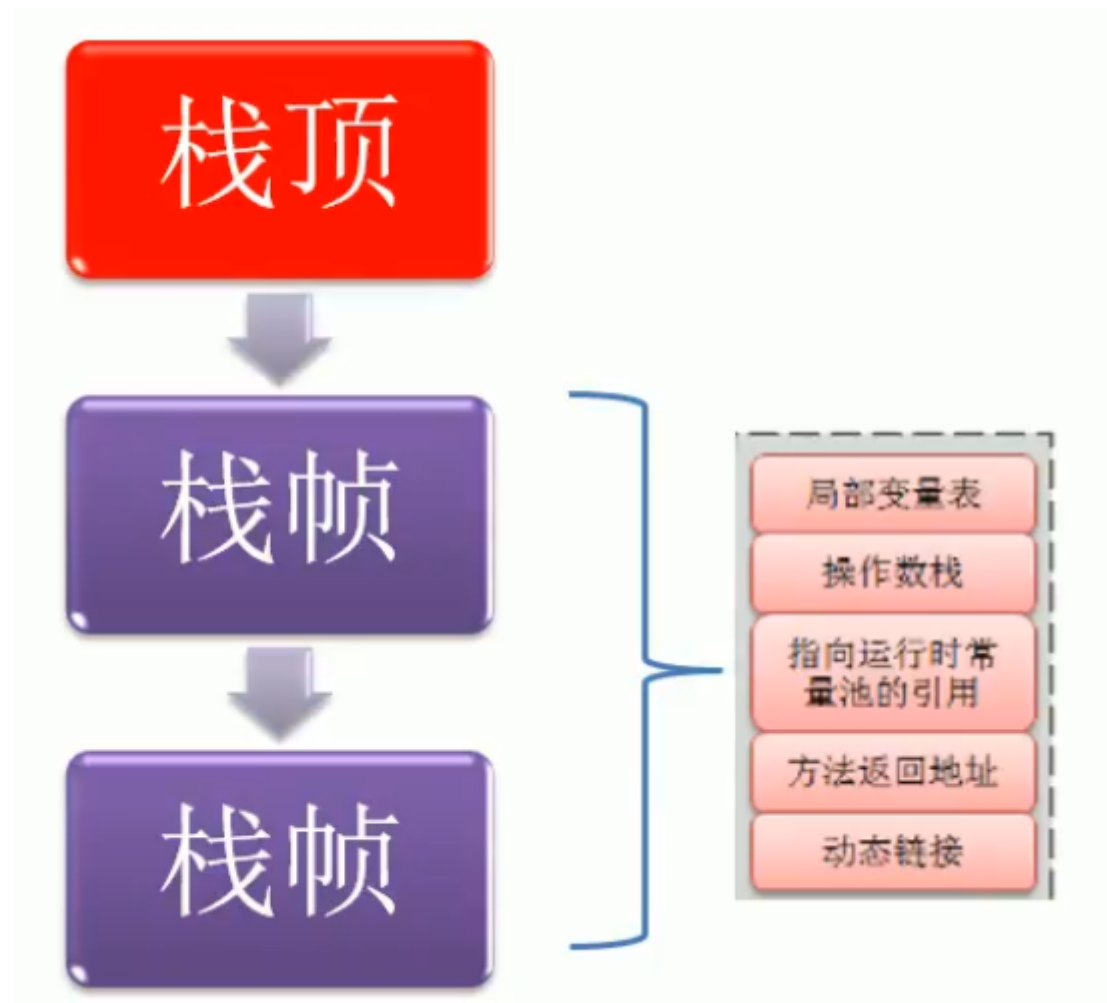
总结

Java 虚拟机栈是理解 Java 方法执行、异常处理、多线程等核心概念的基础。合理设置栈大小、避免栈溢出、理解栈帧结构对于编写高效、健壮的 Java 程序至关重要。

栈主要存储**8种基本类型的变量**、**对象的引用变量**、以及**实例方法**。

这里引出一个名词，**栈帧**，什么是栈帧？

每个方法执行的同时都会创建一个栈帧，用于**存储局部变量表、操作数栈、动态链接、方法出口等信息**，每个方法从调用直至执行完毕的过程，就对应着一个栈帧在虚拟机中入栈到出栈的过程。



简单来说，栈帧对应一个方法的执行和结束，是方法执行过程的内存模型。

其中，栈帧主要保持了3类数据：

本地变量 (Local Variables)：输入参数和输出参数，以及方法内的变量。

栈操作 (Operand Stack)：记录出栈、入栈的操作。

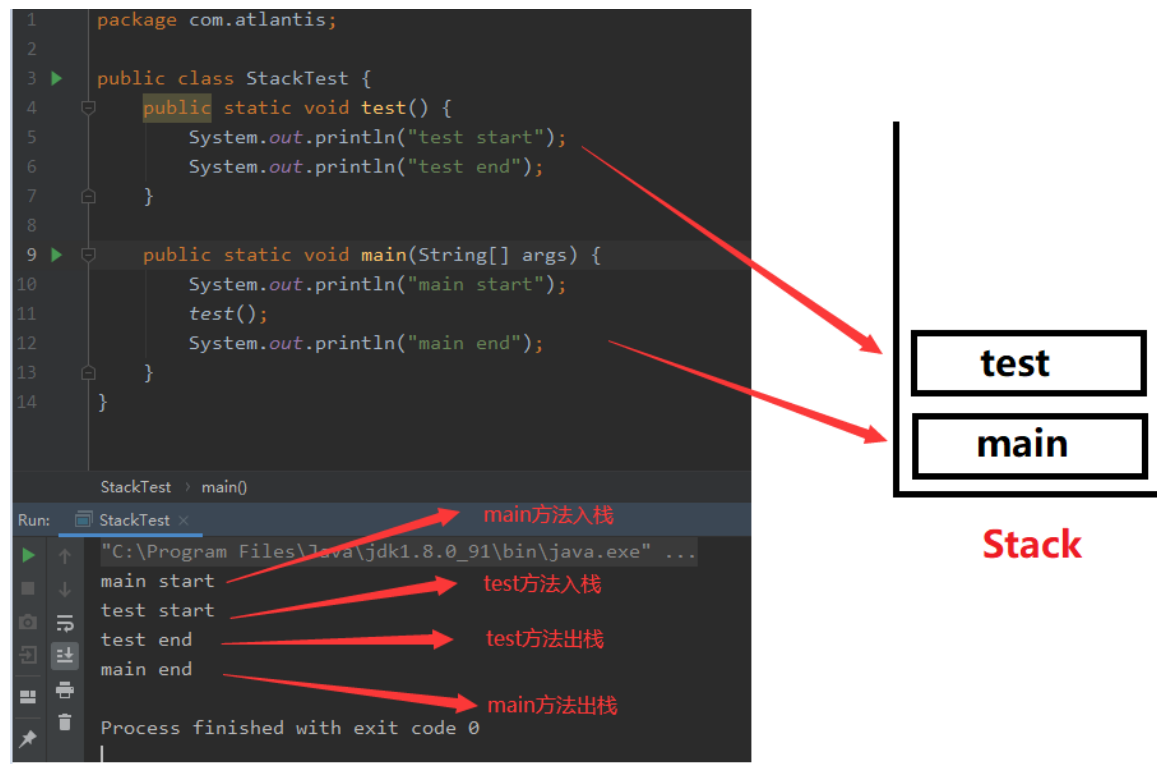
栈帧数据 (Frame Data)：包括类文件、方法等。

1 | 栈的大小是根据JVM有关，一般在256K~756K之间，约等于1Mb左右。

6.2 栈的运行原理

观察下图，在java中，**test()**和**main()**都是方法，而在栈中，称为栈帧。在栈中，main()都是第一个入栈的。

栈的顺序为：main()入栈 --> test()入栈 --> test()出栈 --> main()出栈。



根据代码和运行结果可以知道，main()想要出栈，则必须test()先出栈。那么怎么证明呢？观察下面代码，我们在test()方法中添加了一条语句Thread.sleep(Integer.MAX_VALUE);，来让test()无法进行出栈操作，进而导致main()也无法出栈。运行代码发现，运行结果如我们所料，程序一直停留在test()入栈，无法进行其他操作。

```
1 package com.atlantis;
2
3 public class StackTest {
4     public static void test() throws InterruptedException {
5         System.out.println("test start");
6         Thread.sleep(Integer.MAX_VALUE);
7         System.out.println("test end");
8     }
9
10    public static void main(String[] args) throws InterruptedException {
11        System.out.println("main start");
12        test();
13        System.out.println("main end");
14    }
15 }
```

StackTest > test()

Run: StackTest x

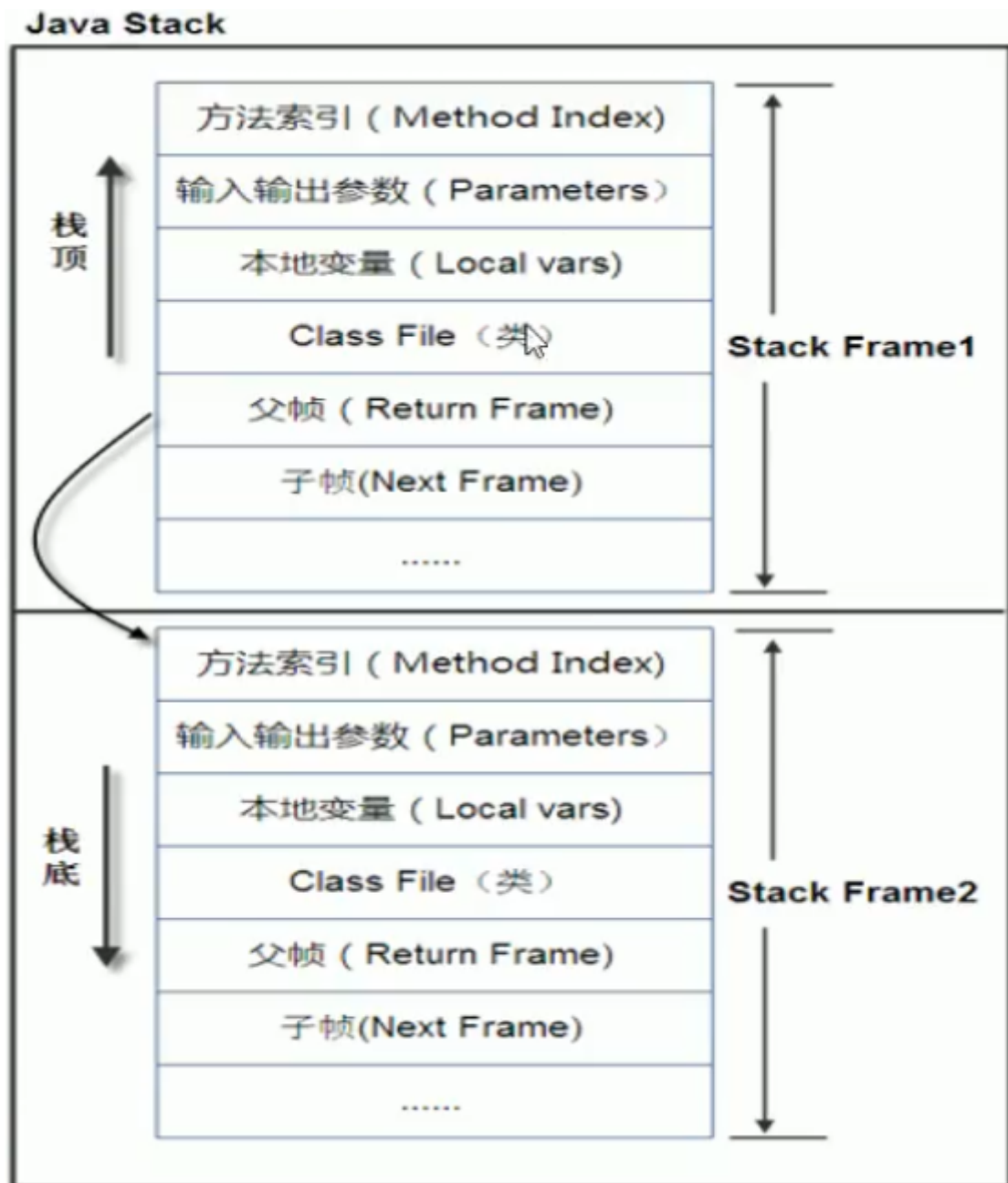
"C:\Program Files\Java\jdk1.8.0_91\bin\java.exe" ...

main start

test start

我们接着观察下图，在图中一个栈中有两个栈帧，分别是Stack Frame1和Stack Frame2，对应方法1和方法2。其中Stack Frame2是最先被调用的方法2，所以它先入栈。然后方法2又调用了方法1，所以Stack Frame1处于栈顶位置。执行完毕后，依次弹出Stack Frame1和Stack Frame2，然后线程结束，栈释放。

所以，每执行一个方法都会产生一个栈帧，并保存到栈的顶部，顶部的栈帧就是当前所执行的方法，该方法执行完毕后会自动出栈。



总结如下，栈中的数据都是以栈帧（Stack Frame）的格式存在，栈帧是一个内存区块，是一个数据集，是一个有关方法（Method）和运行期数据的数据集，当一个方法A被调用时就产生了一个栈帧F1，并被压入到栈中，方法A中又调用了方法B，于是产生栈帧F2也被压入栈中，方法B又调用方法C，于是产生栈帧F3也被压入栈中……执行完毕后，遵循“先进后出，后进先出”的原则，先弹出F3栈帧，再弹出F2栈帧，再弹出F1栈帧。

6.3 栈溢出 StackOverflowError

大家肯定对栈溢出耳熟，那栈溢出是怎么产生的呢？

请看下面代码，test()方法里面又调用了test()方法，即自己调用自己，也叫递归。

同时，栈是一个内存块，它是有大小长度的，而我们观察代码发现，只要代码一运行，test()方法就会一直进行入栈操作，而没有出栈操作，结果肯定会超出栈的大小，进而造成栈溢出错误，即 `java.lang.StackOverflowError`。


```
1 package com.atlantis;
2
3 public class StackTest {
4     public static void test() {
5         System.out.println("test start");
6         test();
7         System.out.println("test end");
8     }
9
10    public static void main(String[] args) {
11        System.out.println("main start");
12        test();
13        System.out.println("main end");
14    }
15 }
```

StackTest > main()

Run: StackTest x

test start
test start
test start
test start
Exception in thread "main" java.lang.StackOverflowError
at sun.nio.cs.UTF_8\$Encoder.encodeLoop(UTF_8.java:691)
at java.nio.charset.CharsetEncoder.encode(CharsetEncoder.java:579)
at sun.nio.cs.StreamEncoder.implWrite(StreamEncoder.java:271)

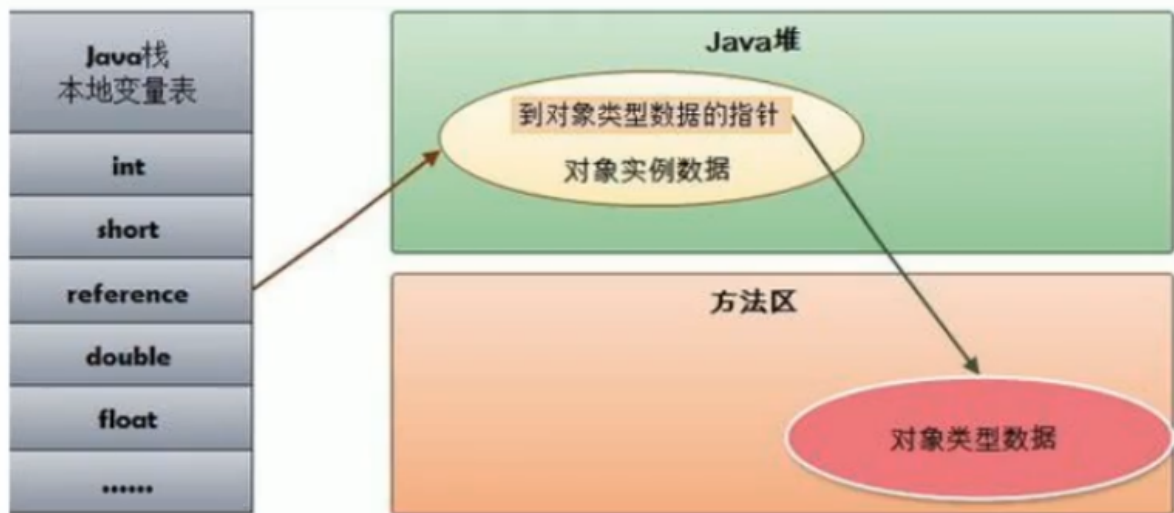
所以说，老哥们，禁止套娃，禁止套娃，禁止套娃！!!!Σ(°Д°ノ)ノ

java.lang.StackOverflowError是错误，不是异常！证明如下：



7.栈、堆、方法区的交互关系

栈、堆、方法区三者的关系如下图，其中reference是引用类型。



举个栗子，比如 `MyObject myObject = new MyObject();`，等号左边 `MyObject myObject` 的 `myObject` 就是引用，在Java栈里面。等号右边的 `new MyObject()`，`new` 出来的 `MyObject` 实例对象在堆里面。简单来说，就是Java栈中的引用 `myObject` 指向了堆中的 `MyObject` 实例对象。

方法区中的对象类型数据，即 `MyObject` 类的模板



8.堆 Heap

8.1 堆体系结构

一个JVM实例只存在一个堆内存，堆内存的大小是可以调节的。类加载器读取了类文件之后，需要把类、方法、常量变量放到堆内存中，保持所以引用类型的真实信息，方便执行器执行。

其中，堆内存分为3个部分：

- Young Generation Space, 新生区、新生代
 - Tenure Generation Space, 老年区、老年代
 - Permanent Space, 永久区、元空间
- Java7之前，堆结构图如下，而Java8则只将永久区变成了**元空间**。

堆内存**逻辑上**分为三部分：新生+养老+永久



总结一下，堆内存**逻辑上**分为新生+养老+永久空间，而堆内存**物理上**分为新生+养老。

8.2 对象在堆中的生命周期

那么如何直观的了解对象在堆中的生命周期呢？

(1) 首先，新生区是类的诞生、成长、消亡的区域。一个类在这里被创建并使用，最后被垃圾回收器收集，结束生命。

(2) 其次，所有的类都是在**Eden Space**被**new**出来的。而当Eden Space的空间用完时，程序又需要创建对象，JVM的垃圾回收器则会将Eden Space中不再被其他对象所引用的对象进行销毁，也就是垃圾回收（**Minor GC**）。此时的GC可以认为是轻量级GC。

(3) 然后将Eden Space中剩余的未被回收的对象，移动到**Survivor 0 Space**，以此往复，直到Survivor 0 Space也满了的时候，再对Survivor 0 Space进行垃圾回收，剩余的未被回收的对象，则再移动到**Survivor 1 Space**。Survivor 1 Space也满了的话，再移动至**Tenure Generation Space**。

(4) 最后，如果Tenure Generation Space也满了的话，那么这个时候就会被垃圾回收（**Major GC or Full GC**）并将该区的内存清理。此时的GC可以认为是重量级GC。如果Tenure Generation Space被GC垃圾回收之后，依旧处于占满状态的话，就会产生我们场景的OOM异常，即**OutOfMemoryError**。

8.3 Minor GC的过程

Survivor 0 Space，幸存者0区，也叫from区；

Survivor 1 Space，幸存者1区，也叫to区。

其中，from区和to区的区分不是固定的，是互相交换的，意思是说，在每次GC之后，两者会进行交换，谁空谁就是to区。

不明白？没关系，接着往下看。



MinorGC的过程（复制->清空->互换）

(1) **Eden Space、from**复制到**to**，年龄+1。

首先，当**Eden Space**满时，会触发第一次GC，把还活着的对象拷贝到**from**区。而当**Eden Space**再次触发GC时，会扫描**Eden Space**和**from**，对这两个区进行垃圾回收，经过此次回收后依旧存活的对象，则直接复制到**to**区（如果对象的年龄已经达到老年的标准，则移动至老年代区），同时把这些对象的年龄+1。

(2) 清空**Eden Space、from**

然后，清空**Eden Space**和**from**中的对象，此时的**from**是空的。

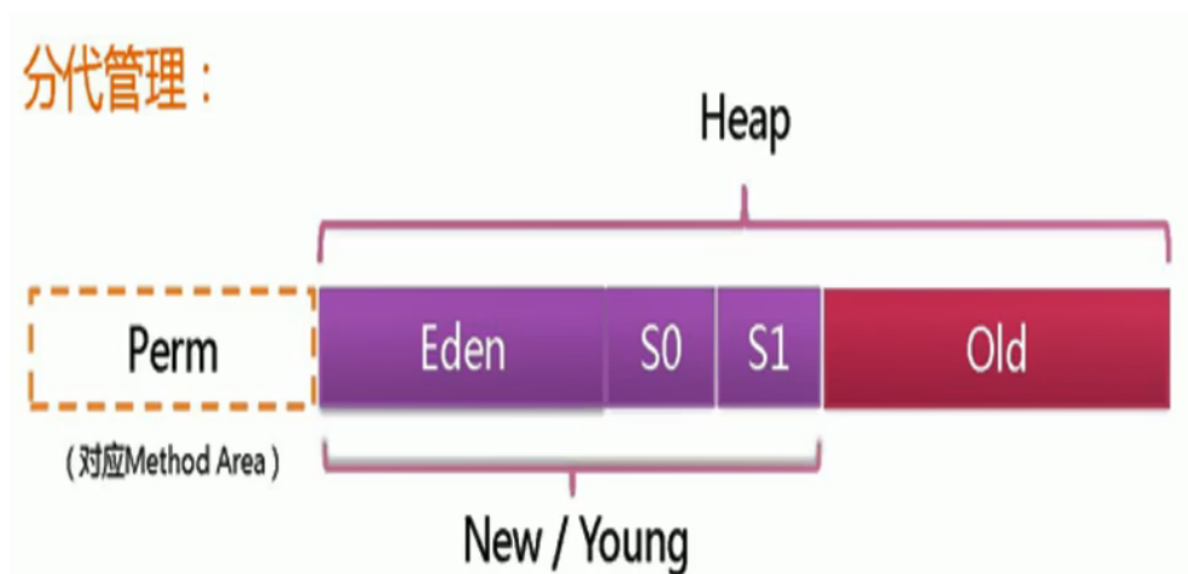
(3) **from**和**to**互换

最后，**from**和**to**进行互换，原**from**成为下一次GC时的**to**，原**to**成为下一次GC时的**from**。部分对象会在**from**和**to**中来回进行交换复制，如果交换15次（由JVM参数**MaxTenuringThreshold**决定，默认15），最终依旧存活的对象就会移动至老年代。

总结一句话，GC之后有交换，谁空谁是**to**。

- 1 这样也是为了保证内存中没有碎片，所以Survivor 0 Space和Survivor 1 Space有一个要是空的。
- 2
- 3 首次GC：Eden → S0（From）
- 4 第二次GC：Eden + S0 → S1（To）→ 交换为S1（From）/S0（To）
- 5 第三次GC：Eden + S1 → S0（To）→ 交换为S0（From）/S1（To）

8.4 HotSpot虚拟机的内存管理



不同对象的生命周期不同，其中98%的对象都是临时对象，即这些对象的生命周期大多只存在于**Eden**区。

实际而言，方法区（**Method Area**）和堆一样，是各个线程共享的内存区域，它用于存储虚拟机加载的：类信息+普通常量+静态常量+编译器编译后的代码等等。虽然JVM规范将方法区描述为堆的一个逻辑部分，但它却还有一个别名叫做**Non-Heap（非堆内存）**，目的就是要和堆区分开。

对于HotSpot虚拟机而言，很多开发者习惯将方法区称为“永久代（Permanent Gen）”。但严格来说两者是不同的，或者说只是使用**永久代来实现方法区**而已，**永久代是方法区（可以理解为一个接口 interface）的一个实现**，JDK1.7的版本中，已经将原本放在永久代的字符串常量池移走。（字符串常量池，JDK1.6在方法区，JDK1.7在堆，JDK1.8在元空间。）



如果没有明确指明，Java虚拟机的名字就叫做HotSpot。

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [版本 10.0.18363.997]
(c) 2019 Microsoft Corporation。保留所有权利。

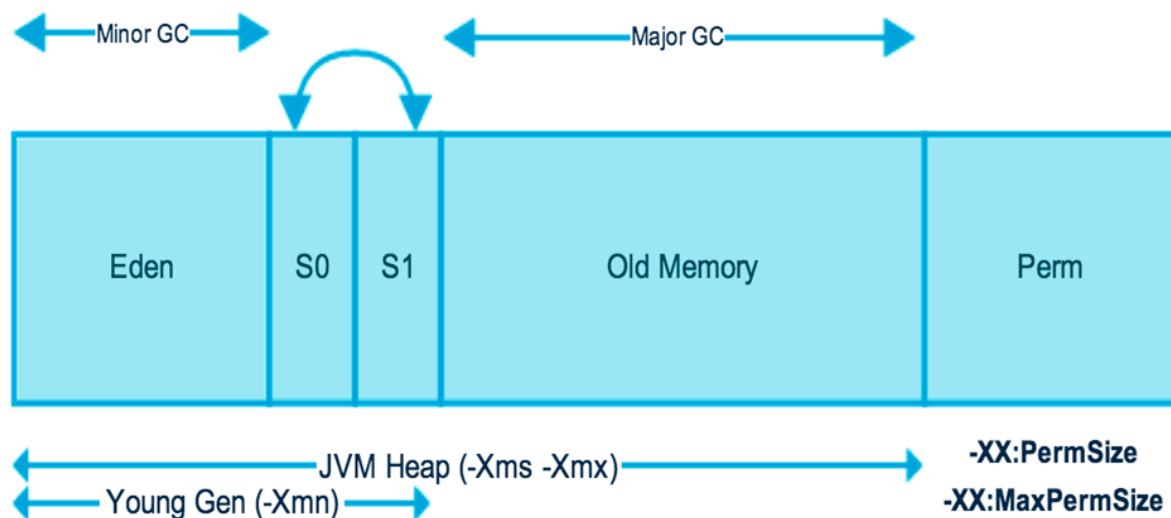
C:\Users\Administrator>java -version
java version "1.8.0_91"
Java(TM) SE Runtime Environment (build 1.8.0_91-b14)
Java HotSpot(TM) 64-Bit Server VM (build 25.91-b14, mixed mode)

C:\Users\Administrator>_
```

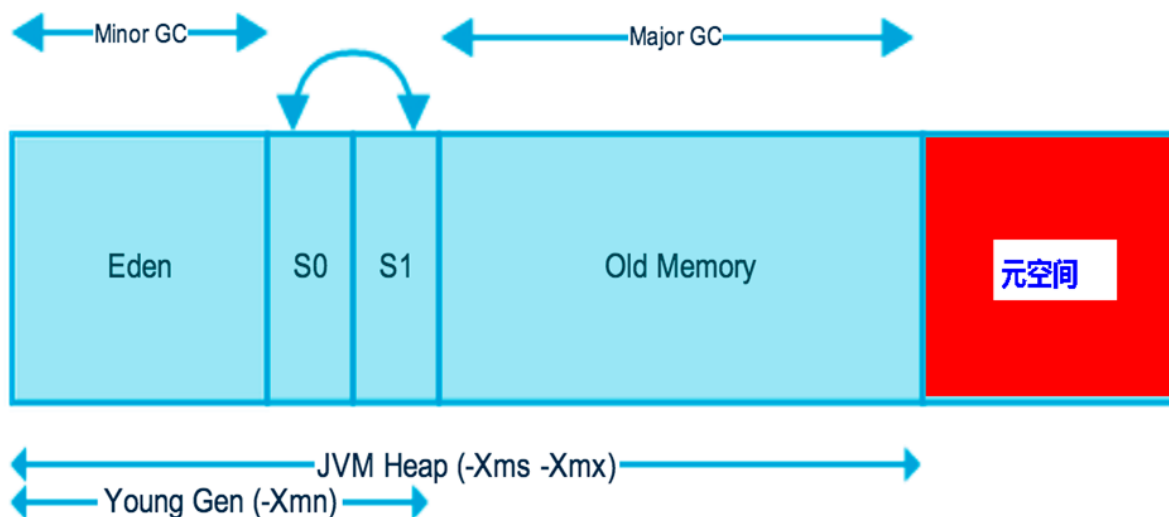
8.5 永久区

永久区是一个常驻内存区域，用于存放JDK自身所携带的**Class**，**Interface**的元数据（也就是上面文章提到的**rt.jar**等），也就是说它存储的是运行环境必须的类信息，被装载进此区域的数据是**不会被垃圾回收器回收掉的**，**关闭VM才会释放**此区域所占用的内存。

(1) JDK1.7



(2) JDK1.8



在JDK1.8中，永久代已经被移除，被一个称为**元空间**的区域所取代。元空间的本质和永久代类似。

元空间与永久代之间最大的区别在于：**永久带使用的JVM的堆内存，但是java8以后的元空间并不在虚拟机中而是使用本机物理内存。**

因此，默认情况下，元空间的大小仅受本地内存限制。

类的元数据放入native memory，字符串池和类的静态变量放入Java堆中，这样可以加载多少类的元数据就不再由MaxPermSize控制, 而由系统的实际可用空间来控制。

方法区、永久代和元空间的区别

1. 方法区 (Method Area)

- **概念**：JVM 规范中定义的一个逻辑区域，用于存储类信息、常量、静态变量、即时编译器编译后的代码等
- **特点**：
 - 是 JVM 规范的一部分
 - 所有 JVM 实现都必须有方法区
 - 规范没有规定具体的实现方式和位置

2. 永久代 (PermGen, Permanent Generation)

- **概念**：HotSpot 虚拟机对方法区的一种实现方式 (JDK 7 及之前)
- **特点**：
 - 位于堆内存中 (非堆)
 - 大小固定，通过-XX:MaxPermSize设置
 - 存储内容：
 - 类信息
 - 运行时常量池
 - 字符串常量池 (JDK 6 及之前)
 - 类静态变量
 - JIT 编译后的代码
- **问题**：
 - 容易发生java.lang.OutOfMemoryError: PermGen space
 - 大小难以预测，调优困难

3. 元空间 (Metaspace)

- **概念**: JDK 8 引入, 替代永久代的方法区实现
- **特点**:
 - 使用本地内存 (Native Memory) 而非 JVM 堆内存
 - 默认不限制大小 (受限于系统内存)
 - 通过-XX:MaxMetaspaceSize设置上限
 - 存储内容:
 - 类元数据
 - 运行时常量池
 - **字符串常量池 (移至堆中)**
 - **类静态变量 (移至堆中)**
- **优势**:
 - 避免永久代内存溢出
 - 自动调整大小
 - 提高 GC 效率
 - 简化 Full GC 过程

三者的关系

```
1 | JVM 规范
2 |   ↓
3 | 方法区 (逻辑概念)
4 |   ├── 永久代 (JDK 7 及之前, HotSpot 实现)
5 |   └── 元空间 (JDK 8+, HotSpot 实现)
```

重要变化

1. JDK 7:

- 将字符串常量池从永久代移动到了堆中

2. JDK 8:

- 完全移除了永久代
- 引入元空间
- 将类静态变量和字符串常量池移到了堆中

相关 JVM 参数

• 永久代:

- -XX:PermSize: 初始大小

- 1 | -XX:MaxPermSize

: 最大大小

• 元空间:

- 1 | -XX:MetaspaceSize

: 初始大小

- 1 | -XX:MaxMetaspaceSize

: 最大大小 (默认无限制)

o 1 | -XX:MinMetaspaceFreeRatio

: GC 后最小元空间空闲比例

o 1 | -XX:MaxMetaspaceFreeRatio

: GC 后最大元空间空闲比例

总结

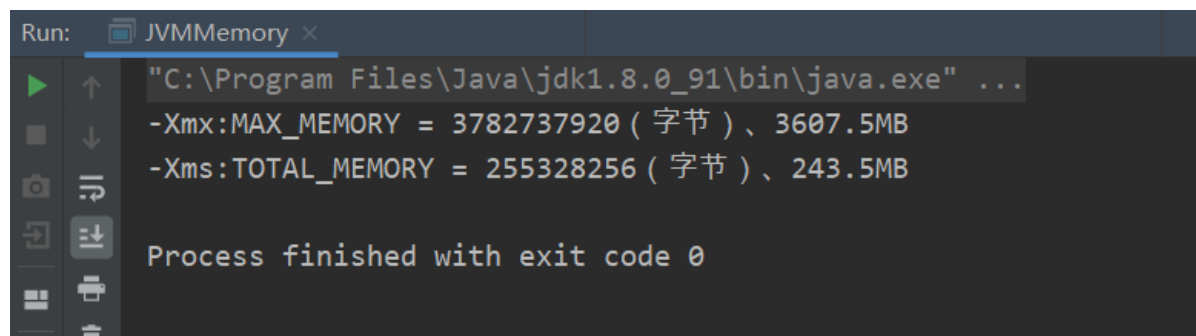
- **方法区**是 JVM 规范中的概念
- **永久代**是方法区在 JDK 7 及之前的实现
- **元空间**是 JDK 8 及之后方法区的实现
- 元空间使用本地内存, 避免了永久代的内存溢出问题, 并提供了更好的性能

8.6 堆参数调优

在进行堆参数调优前, 我们可以通过下面的代码来获取虚拟机的相关内存信息。

```
1 public class JVMMemory {
2     public static void main(String[] args) {
3         // 返回 Java 虚拟机试图使用的最大内存量
4         long maxMemory = Runtime.getRuntime().maxMemory();
5         System.out.println("MAX_MEMORY = " + maxMemory + " (字节)、" +
6             (maxMemory / (double) 1024 / 1024) + "MB");
7         // 返回 Java 虚拟机中的内存总量
8         long totalMemory = Runtime.getRuntime().totalMemory();
9         System.out.println("TOTAL_MEMORY = " + totalMemory + " (字节)、" +
10            (totalMemory / (double) 1024 / 1024) + "MB");
11     }
12 }
```

运行结果如下:

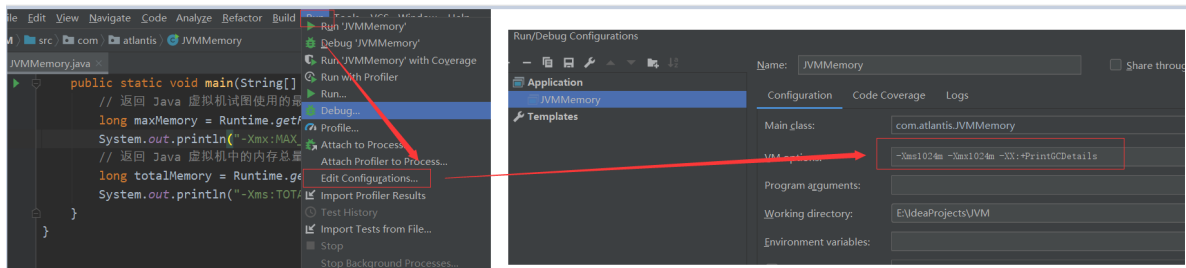


```
Run: JVMMemory x
"C:\Program Files\Java\jdk1.8.0_91\bin\java.exe" ...
-Xmx:MAX_MEMORY = 3782737920 ( 字节 )、3607.5MB
-Xms:TOTAL_MEMORY = 255328256 ( 字节 )、243.5MB
Process finished with exit code 0
```

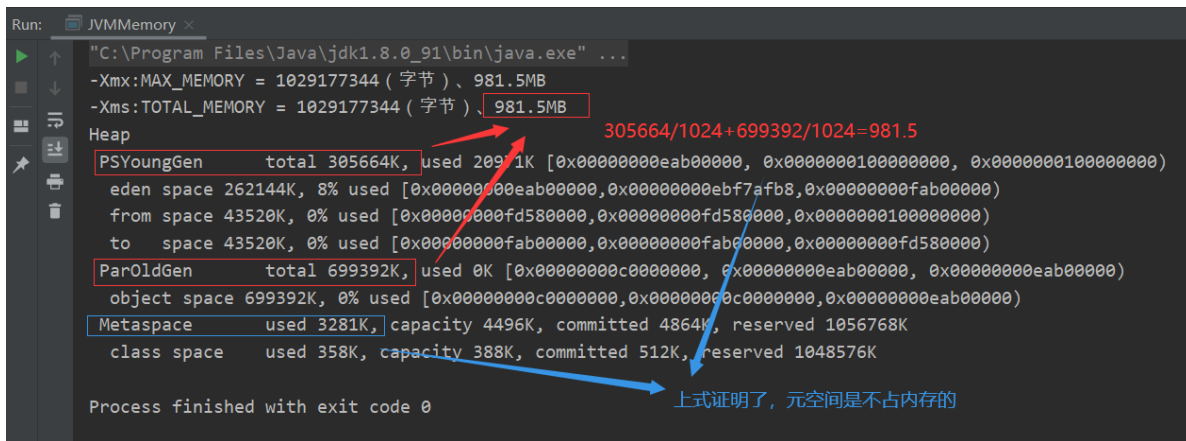
有人就有疑问了, 这个3607.5MB和243.5MB是怎么算出来的? 看下图就明白了, 虚拟机**最大内存**为物理内存的**1/4**, 而**初始分配的内存**为物理内存的**1/64**。

-Xms	设置初始分配大小, 默认为物理内存的“1 / 64”
-Xmx	最大分配内存, 默认为物理内存的 “1 / 4”
-XX:+PrintGCDetails	输出详细的GC处理日志

IDEA中如何配置JVM内存参数? 在【Run】->【Edit Configuration...】->【VM options】中, 输入参数-Xms1024m -Xmx1024m -XX:+PrintGCDetails, 然后保存退出。



运行结果如下：



- 1 | JVM的初始内存和最大内存一般怎么配？
- 2 | 答：初始内存和最大内存一定是一样大，理由是避免GC和应用程序争抢内存，进而导致内存忽高忽低产生停顿。

8.7 堆溢出 OutOfMemoryError

现在我们来演示一下OOM，首先把堆内存调成10M后，再一直new对象，导致Full GC也无法处理，直至撑爆堆内存，进而导致OOM堆溢出错误，程序及结果如下：

```
1 | import java.util.Random;
2 | public class OOMTest {
3 |     public static void main(String[] args) {
4 |         String str = "Atlantis";
5 |         while (true) {
6 |             // 每执行下面语句，会在堆里创建新的对象
7 |             str += str + new Random().nextInt(88888888) + new
Random().nextInt(999999999);
8 |         }
9 |     }
10 | }
```

```
Run: OOMTest x
"C:\Program Files\Java\jdk1.8.0_91\bin\java.exe" ...
[GC (Allocation Failure) [PSYoungGen: 2018K->504K(2560K)] 2018K->768K(9728K), 0.0013088 secs] [Times: user=0.00 sys=0.00, re
[GC (Allocation Failure) [PSYoungGen: 2208K->497K(2560K)] 2472K->1445K(9728K), 0.0010611 secs] [Times: user=0.00 sys=0.00, r
[GC (Allocation Failure) [PSYoungGen: 2485K->432K(2560K)] 3433K->2540K(9728K), 0.0007017 secs] [Times: user=0.00 sys=0.00, r
[GC (Allocation Failure) [PSYoungGen: 2032K->432K(2560K)] 5689K->4862K(9728K), 0.0010703 secs] [Times: user=0.00 sys=0.00, r
[Full GC (Ergonomics) [PSYoungGen: 1239K->0K(2560K)] [ParOldGen: 5978K->2960K(7168K)] 7217K->2960K(9728K), [Metaspace: 3289K
[Full GC (Ergonomics) [PSYoungGen: 1584K->0K(2560K)] [ParOldGen: 6056K->2183K(7168K)] 7640K->2183K(9728K), [Metaspace: 3289K
[GC (Allocation Failure) [PSYoungGen: 1588K->0K(2560K)] 6867K->5278K(9728K), 0.0010467 secs] [Times: user=0.00 sys=0.00, rea
[GC (Allocation Failure) [PSYoungGen: 0K->0K(1536K)] 5278K->5278K(8704K), 0.0006093 secs] [Times: user=0.00 sys=0.00, real=6
[Full GC (Allocation Failure) [PSYoungGen: 0K->0K(1536K)] [ParOldGen: 5278K->5278K(7168K)] 5278K->5278K(8704K), [Metaspace:
[GC (Allocation Failure) [PSYoungGen: 0K->0K(2048K)] 5278K->5278K(9216K), 0.0005277 secs] [Times: user=0.00 sys=0.00, real=6
[Full GC (Allocation Failure) [PSYoungGen: 0K->0K(2048K)] [ParOldGen: 5278K->5258K(7168K)] 5278K->5258K(9216K), [Metaspace:
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
    at java.util.Arrays.copyOf(Arrays.java:3332)
    at java.lang.AbstractStringBuilder.expandCapacity(AbstractStringBuilder.java:137)
    at java.lang.AbstractStringBuilder.ensureCapacityInternal(AbstractStringBuilder.java:121)
    at java.lang.AbstractStringBuilder.append(AbstractStringBuilder.java:617)
    at java.lang.StringBuilder.append(StringBuilder.java:208)
    at com.atlantis.OOMTest.main(OOMTest.java:10)
先GC回收, 再Full GC回收, 直到Full GC回收后满了, 就报OOM
```

如果出现java.lang.OutOfMemoryError: Java heap space异常, 说明Java虚拟机的堆内存不够, 造成堆内存溢出。原因有两点:

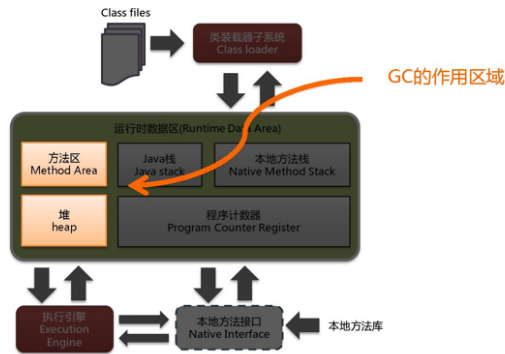
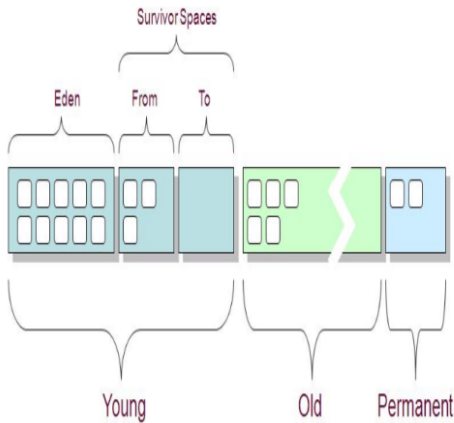
- ①Java虚拟机的堆内存设置太小, 可以通过参数-Xms和-Xmx来调整。
- ②代码中创建了大量对象, 并且长时间不能被GC回收 (存在被引用)。

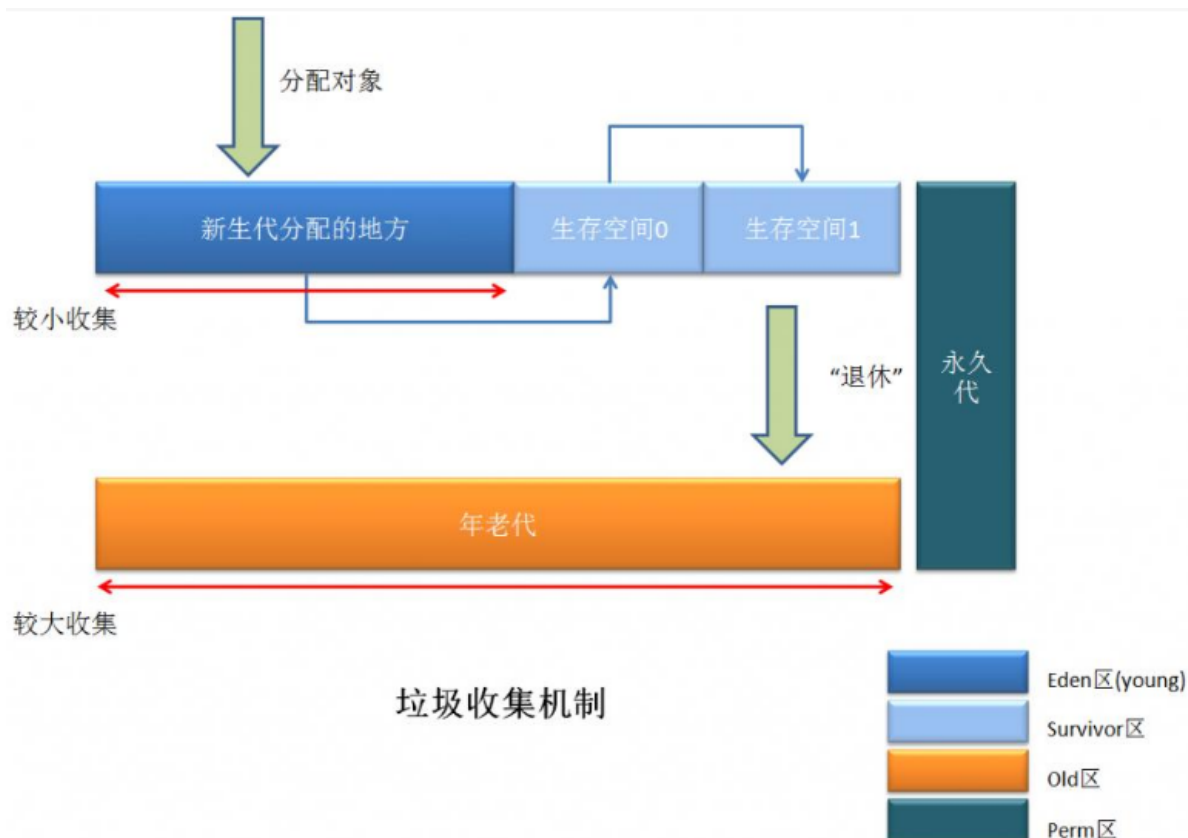
9.GC (Java Garbage Collection)

9.1 GC垃圾收集机制

对于GC垃圾收集机制, 我们需要记住以下几点:

- 1. 次数上频繁收集Young区。
- 2. 次数上较少收集Old区。
- 3. 基本不动元空间。





JVM在进行GC时，并非每次都对上面三个内存区域一起回收的，大部分时候回收的都是指新生代。因此GC按照回收的区域又分了两类型，一种是普通GC（minor GC），一种是全局GC（major GC or Full GC）

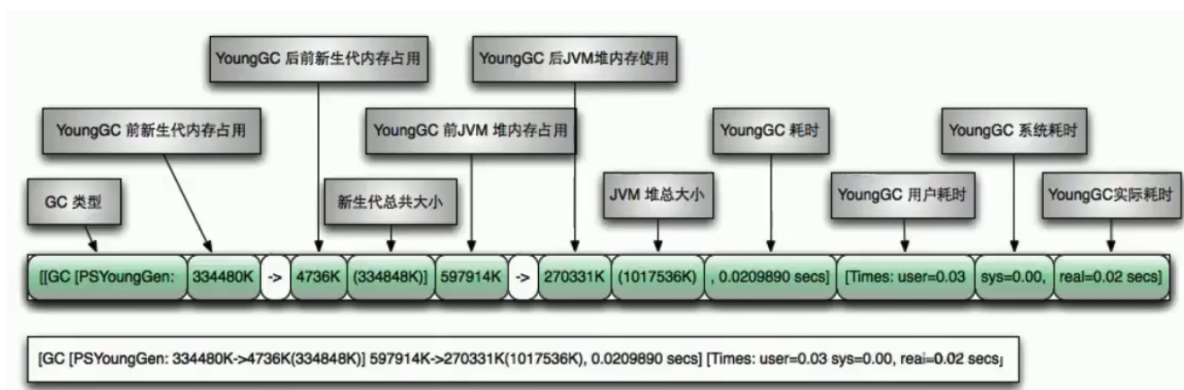
Minor GC和Full GC的区别：

- 普通GC（minor GC）**：只针对新生代区域的GC，指发生在新生代的垃圾收集动作，因为大多数Java对象存活率都不高，所以Minor GC非常频繁，一般回收速度也比较快。
- 全局GC（major GC or Full GC）**：指发生在老年代的垃圾收集动作，出现了Major GC，经常会伴随至少一次的Minor GC（但并不是绝对的）。Major GC的速度一般要比Minor GC慢上10倍以上

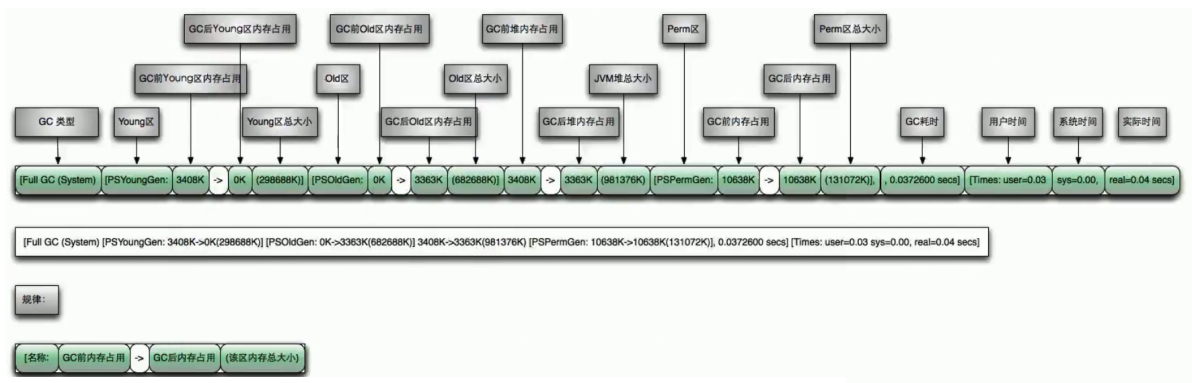
9.2 GC日志信息详解

通过上面OOM案例，是不是觉得那一大片的日志信息看不懂？懵逼？没事，通过下图你就知道如何阅读GC日志信息。

(1) YGC相关参数：



(2) FGC相关参数：



9.3 GC四大算法

复制算法 (Copying)

标记清除 (Mark-Sweep)

标记压缩 (Mark-Compact)

分代收集算法

具体内容请跳转至：GC四大算法详解

版权声明：本文为CSDN博主「AtlantisChina」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：<https://blog.csdn.net/q961250375/article/details/107499173>

10.如何判断Java中对象是否存活？

10.1 引用计数算法

引用计数算法是给每个对象设置一个计数器，当有地方引用这个对象的时候，计数器+1，当引用失效的时候，计数器-1，当计数器为0的时候，JVM就认为该对象不再被使用，是“垃圾”了。

引用计数实现简单，效率高；但是不能解决循环引用问题（A对象引用B对象，B对象又引用A对象，但是A，B对象已不被任何其他对象引用），同时每次计数器的增加和减少都带来了许多额外的开销，所以在JDK1.1之后，这个算法已经不再使用了。



缺点：

- 每次对对象赋值时均要维护引用计数器，且计数器本身也有一定的消耗；
- 较难处理循环引用

JVM的实现一般不采用这种方式

10.2 根搜索方法

根搜索方法是通过一些GCRoots对象作为起点，从这些节点开始往下搜索，搜索通过的路径成为引用链（ReferenceChain），当一个对象没有被GCRoots的引用链连接的时候，说明这个对象是不可用的。

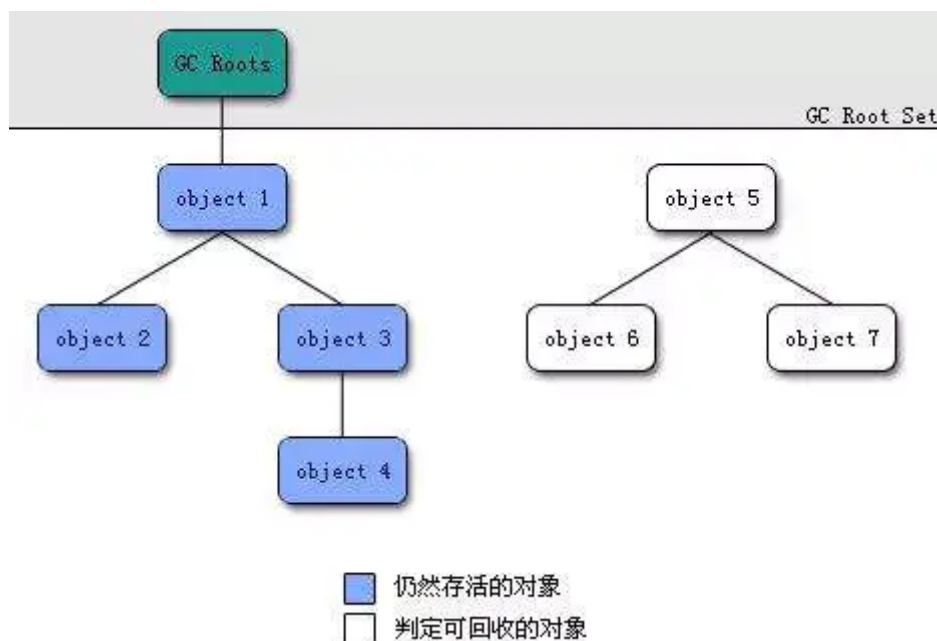
GCRoots对象包括：

虚拟机栈（栈帧中的本地变量表）中的引用的对象。

方法区域中的类静态属性引用的对象。

方法区域中常量引用的对象。

方法栈中JNI（**Native方法**）的引用的对象。



11.复制算法（Copying）：适用于新生代

11.1 原理分析

虚拟机把新生代分为了三部分：1个**Eden**区和2个**Survivor**区（分别叫**from**和**to**），默认比例为8:1:1。

一般情况下，新创建的对象都会被分配到**Eden**区（一些大对象特殊处理），这些对象经过第一次**Minor GC**后，如果仍然存活，将会被移到**Survivor**区。对象在**Survivor**区中每熬过一次Minor GC，年龄+1，当它的年龄增加到一定程度时（默认是15，通过-XX:MaxTenuringThreshold来设定参数），就会被移动到老年代中。

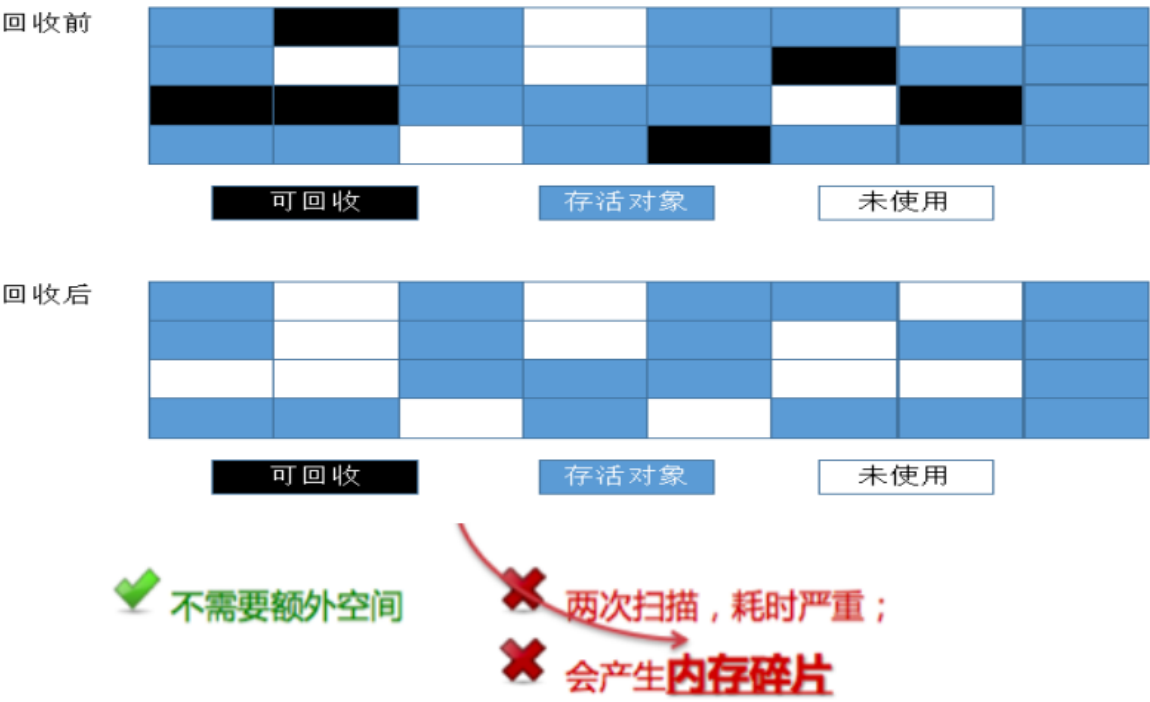
因为新生代中的对象基本都是**朝生夕死**（被GC回收率90%以上），所以在新生代的垃圾回收算法使用的是**复制算法**。

复制算法的基本思想就是将内存分为两块，每次只用其中一块（**from**），当这一块内存用完，就将还活着的对象复制到另外一块上面。

我们来举个栗子，在GC开始的时候，对象只会存在于Eden区和名为from的Survivor区，Survivor区to是空的。紧接着进行GC，Eden区中所有存活的对象都会被复制到to，而在from区中，仍存活的对象会根据他们的年龄值来决定去向。年龄达到一定值（默认15）的对象会被移动到老年代中，没有达到阈值的对象会被复制到to区域。经过这次GC后，Eden区和from区已经被清空。这个时候，from和to会交换他们的角色，也就是新的to就是上次GC前的from，新的from就是上次GC前的to。不管怎样，都会保证名为to的Survivor区域是空的。Minor GC会一直重复这样的过程，直到to区被填满，to区被填满之后，会将所有对象移动到老年代中。

12.1 原理分析

标记清除算法，主要分成标记和清除两个阶段，先标记出要回收的对象，然后统一回收这些对象，如下图：



简单来说，标记清除算法就是当程序运行期间，若可以使用的内存被耗尽的时候，GC线程就会被触发并将程序暂停，随后将要回收的对象标记一遍，最终统一回收这些对象，完成标记清理工作接下来便让应用程序恢复运行。

主要进行两项工作，第一项则是标记，第二项则是清除。

- **标记**：从引用根节点开始标记遍历所有的GC Roots，先标记出要回收的对象。
- **清除**：遍历整个堆，把标记的对象清除。



12.2 优缺点

优点：不需要额外的内存空间。

缺点：需要暂停整个应用，会产生内存碎片；两次扫描，耗时严重。

简单来说，它的缺点就是**效率比较低**（递归与全堆对象遍历），而且在进行GC的时候，需要停止应用程序，这会导致用户体验非常差劲。

而且这种方式**清理出来的空闲内存是不连续的**，这点不难理解，我们的死亡对象都是随机分布在内存当中，现在把它们清除之后，内存的布局自然会零碎不连续。而为了应付这一点，JVM就不得不维持一个内存的空闲列表，这又是一种开销。并且在分配数组对象的时候，需要去内存寻找连续的内存空间，但此时的内存空间太过零碎分散，因此资源耗费加大。

13.标记压缩 (Mark-Compact) ： 适用于老年代

13.1 原理分析

简单来说，就是**先标记**，后整理，如下图所示：

1. 标记 (Mark)：

与 **标记-清除** 一样。



2. 压缩 (Compact)：

再次扫描，并往一端 **滑动** 存活对象。



✔ 没有内存碎片，可以利用bump the-pointer ✖ 需要移动对象的成本

Mark Sweep 0. 初始状态					

13.2 优缺点

优点： 没有内存碎片。

缺点： 需要移动对象的成本，效率也不高（不仅要标记所有存活对象，还要整理所有存活对象的引用地址）。

13.3 标记清除压缩 (Mark-Sweep-Compact)

标记-清除-压缩 (Mark-Sweep-Compact)

原理：

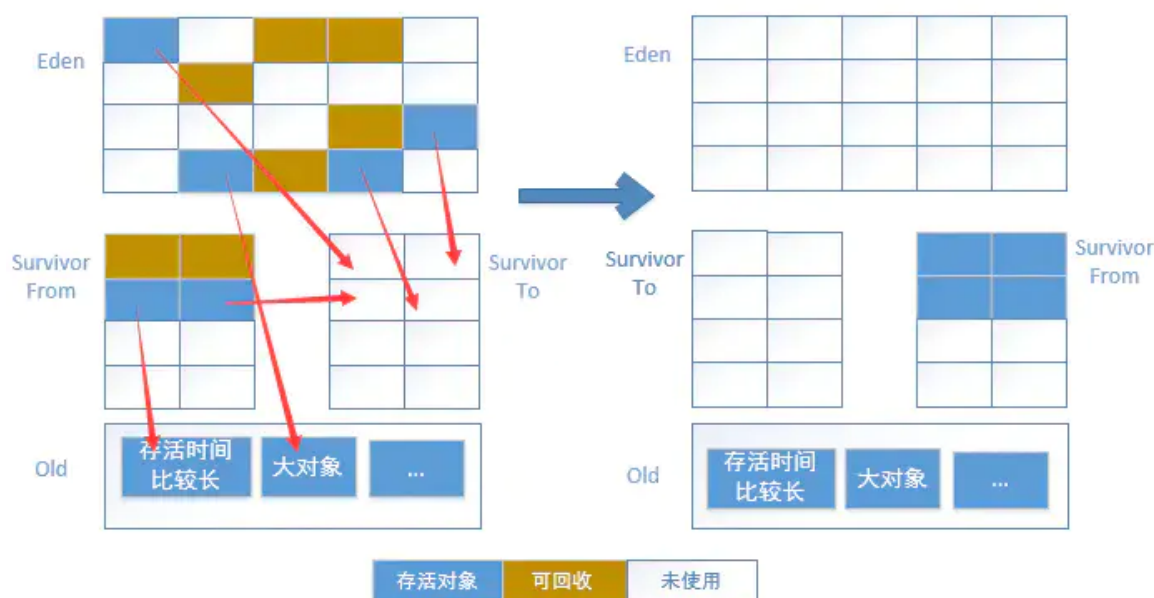
1. Mark-Sweep 和 Mark-Compact的结合。
2. 和Mark-Sweep一致，当进行多次GC后才Compact。

✓ 减少移动对象的成本

14.分代收集算法

当前商业虚拟机都是采用分代收集算法，它根据对象存活周期的不同将内存划分为几块，一般是把Java堆分为新生代和老年代，然后根据各个年代的特点采用最适当的垃圾收集算法。

在新生代中，每次垃圾收集都发现有大批对象死去，只有少量存活，就选用**复制算法**，而老年代因为对象存活率高，没有额外空间对它进行分配担保，就必须使用**标记清除**或者**标记压缩**算法来进行回收。



15.总结

15.1 年轻代 (Young Gen)

年轻代特点是内存空间相对老年代较小，对象存活率低。

复制算法的效率只和当前存活对象大小有关，因而很适用于年轻代的回收。而复制算法的内存利用率不高的问题，可以通过虚拟机中的两个**Survivor**区设计得到缓解。

15.2 老年代 (Tenure Gen)

老年代的特点是内存空间较大，对象存活率高。

这种情况，存在大量存活率高的对象，复制算法明显变得不合适。一般是由标记清除或者是标记清除与标记整理的混合实现。

(1) **标记阶段 (Mark)** 的开销与存活对象的数量成正比。这点上说来，对于老年代，标记清除或者标记整理有一些不符，但可以通过多核/线程利用，对并发、并行的形式提标记效率。

(2) **清除阶段 (Sweep)** 的开销与所管理内存空间大小成正比。但Sweep“就地处决”的特点，回收的过程**没有对象的移动**。使其相对其他有对象移动步骤的回收算法，仍然是效率最好的。但是**需要解决内存碎片问题**。

(3) **整理阶段 (Compact)** 的开销与存活对象的数据成正比。如上一条所描述，**对于大量对象的移动是很大开销的**，做为老年代的第一选择并不合适。

基于上面的考虑，老年代一般是由**标记清除**或者是**标记清除与标记整理的混合实现**。以虚拟机中的CMS回收器为例，CMS是基于**Mark-Sweep**实现的，对于对象的回收效率很高。而对于碎片问题，CMS采用基于**Mark-Compact**算法的Serial Old回收器做为补偿措施：当内存回收不佳（碎片导致的**Concurrent Mode Failure**时），将采用Serial Old执行**Full GC**以达到对老年代内存的整理。

1 | 参考文章：OMM Error和七大垃圾回收器详解 <https://www.jianshu.com/p/ba415aa2330b>

16.附录.常见面试问题

16.1 GC四种算法哪个好？

没有哪个算法是能一次性解决所有问题的，因为JVM垃圾回收使用的是分代收集算法，没有最好的算法，只有根据每一代他的垃圾回收的特性用对应的算法。例如新生代使用复制算法，老年代使用标记清除和标记整理算法。

所以说，没有最好的垃圾回收机制，只有最合适的。

16.2 请说出各个垃圾回收算法的优缺点

(1) **内存效率**：复制算法 > 标记清除算法 > 标记整理算法（此处的效率只是简单的对比时间复杂度，实际情况不一定如此）。

(2) **内存整齐度**：复制算法 = 标记整理算法 > 标记清除算法。

(3) **内存利用率**：标记整理算法 = 标记清除算法 > 复制算法。

可以看出，效率上来说，复制算法是当之无愧的老大，但是却浪费了太多内存，而为了尽量兼顾上面所提到的三个指标，标记整理算法相对来说更平滑一些，但效率上依然不尽如人意，它比复制算法多了一个标记的阶段，又比标记清除多了一个整理内存的过程。

版权声明：本文为CSDN博主「AtlantisChina」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：<https://blog.csdn.net/q961250375/article/details/107859902>